

Stat4307 Lab3

Kent Roller

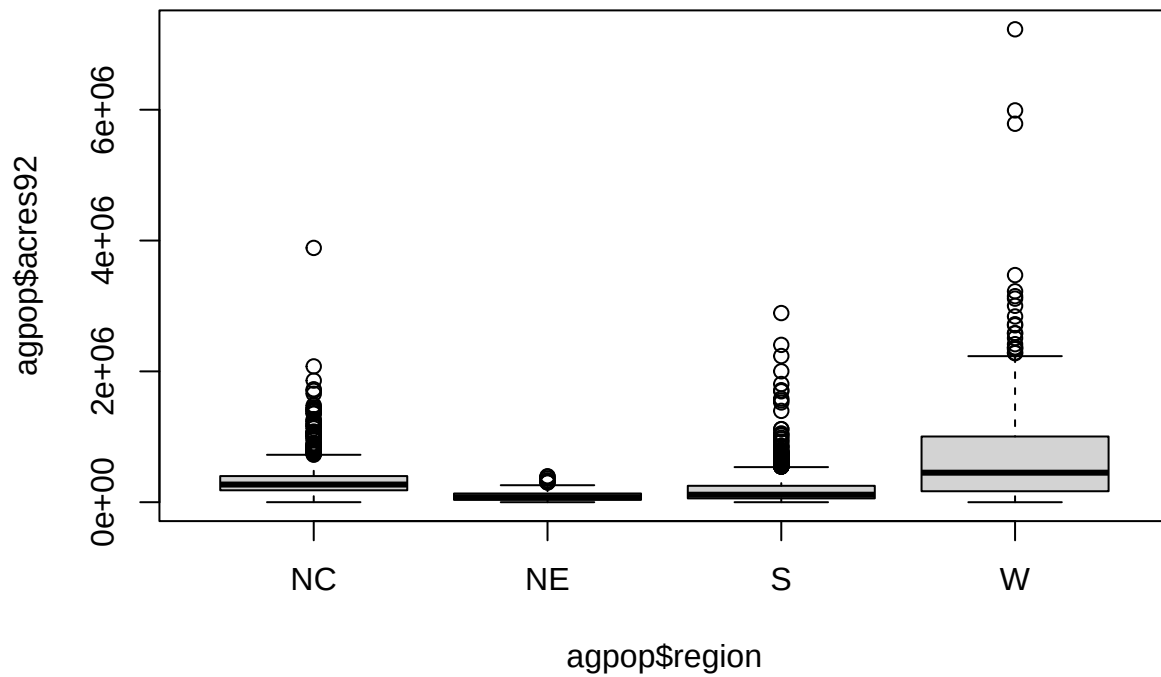
2023-02-16

1. Download the agpop.csv and read it into R...

```
#file.choose()  
agpop<-read.csv("C:\\Users\\super\\Downloads\\agpop.csv", na.strings="-99")
```

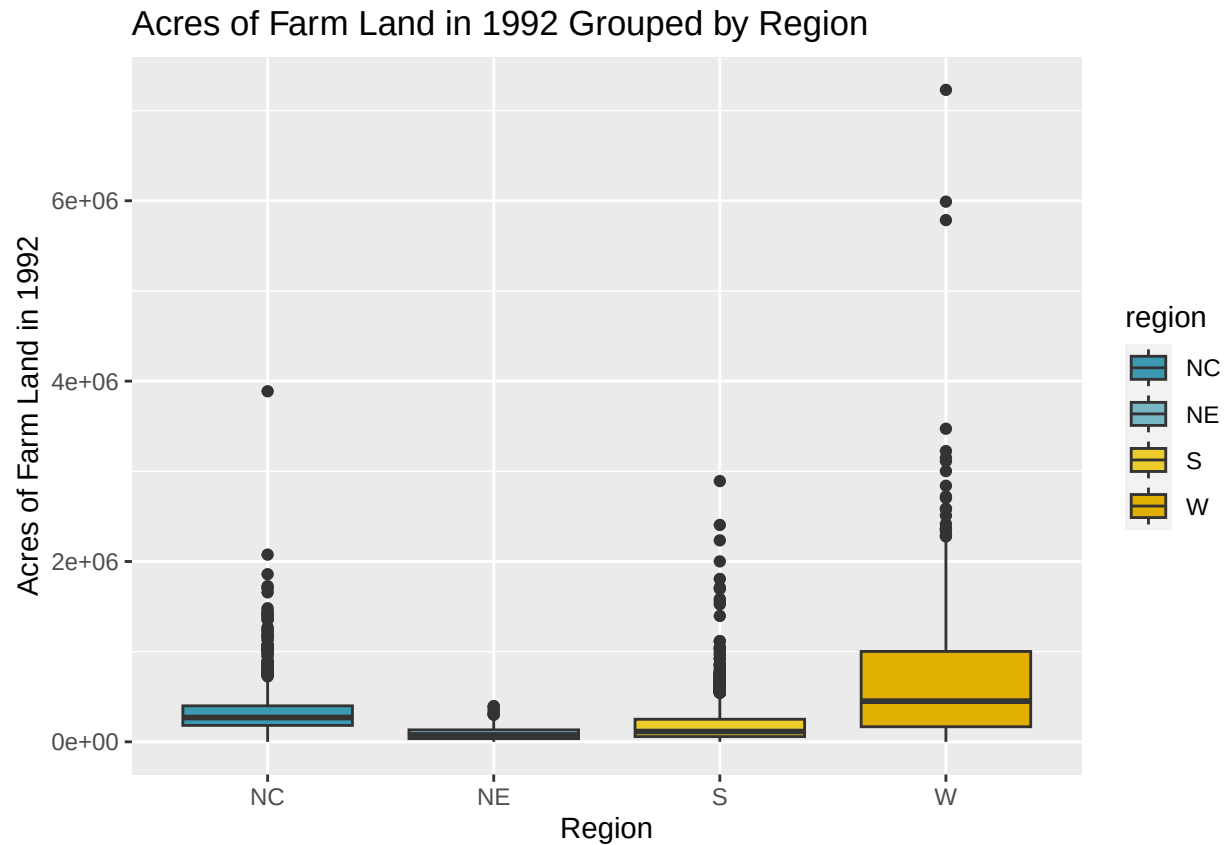
2. Create a boxplot of the values...

```
boxplot(agpop$acres92~agpop$region)  
library(wesanderson)  
library(ggplot2)
```



```
ggplot(agpop,aes(x=region,y=acres92))+ggtitle("Acres of Farm Land in 1992 Grouped by Region")+labs(y="Acres of Farm Land in 1992")
```

```
## Warning: Removed 19 rows containing non-finite values ('stat_boxplot()').
```



3. Select an SRS of size 300

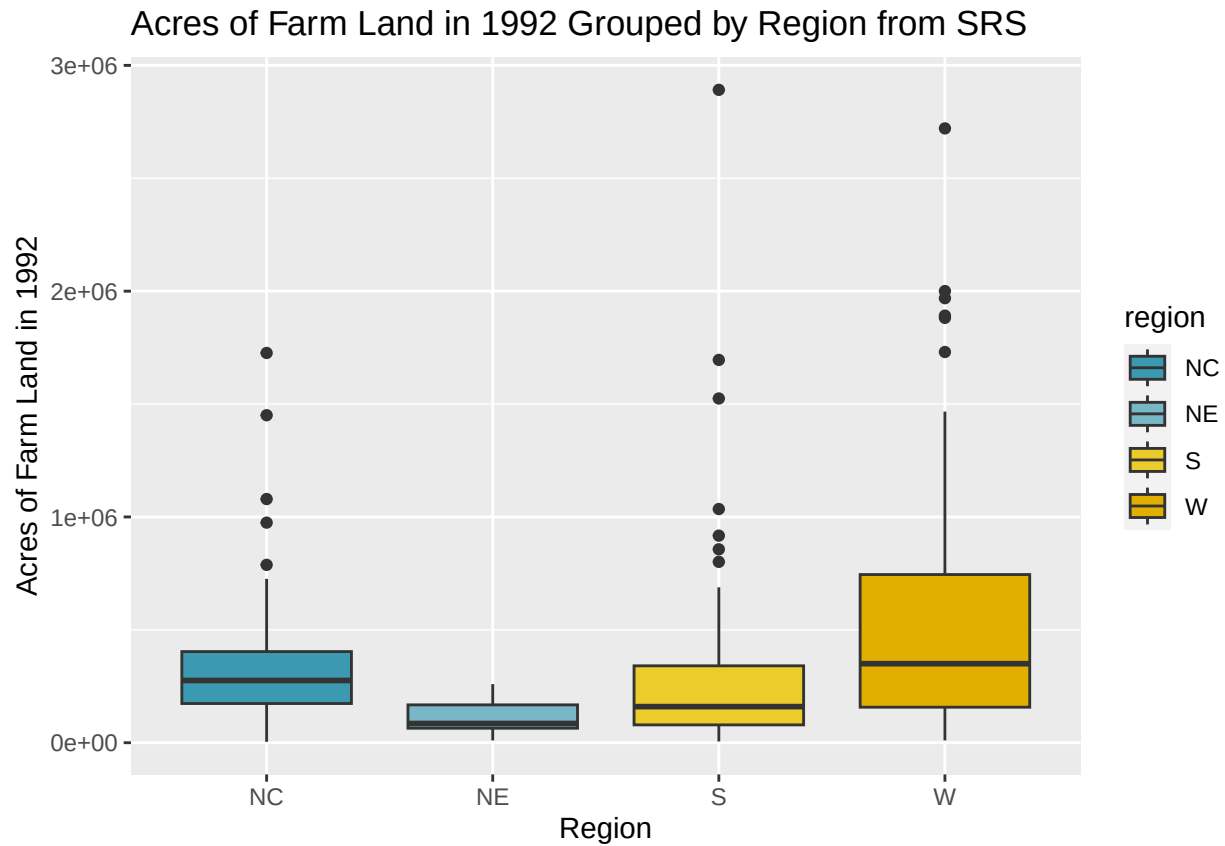
```
set.seed(7842)
obs<-sample(1:nrow(agpop),300)
srs<-agpop[obs,]
head(srs)
```

```
##          county state acres92 acres87 acres82 farms92 farms87 farms82
## 2959  KEWAUNEE COUNTY  WI  170228  183626  182056    893    991   1124
## 1042   MADISON COUNTY  KY  247266  245581  237974   1575   1597   1712
## 2988  SHEBOYGAN COUNTY  WI  207128  209508  217888   1071   1213   1349
## 807    JAY COUNTY     IN  182836  188637  204345    852    922   1062
## 1082   WAYNE COUNTY    KY  135850  136970  140014    889    979   1082
## 773    BENTON COUNTY   IN  270618  271516  256225    500    614    628
##      largef92 largef87 largef82 smallf92 smallf87 smallf82 region
## 2959         3         5         3        36        30        37    NC
## 1042        23        19        14       225       206       254     S
## 2988        18        13         8        93       119       128    NC
## 807         36        24        15        60        68        59    NC
## 1082        12         9         8       124       130       146     S
## 773         74        59        44        29        24        32    NC
```

4. Create a boxplot of the values from the SRS

```
ggplot(srs,aes(x=region,y=acres92))+ggtitle("Acres of Farm Land in 1992 Grouped by Region from SRS")+lab
```

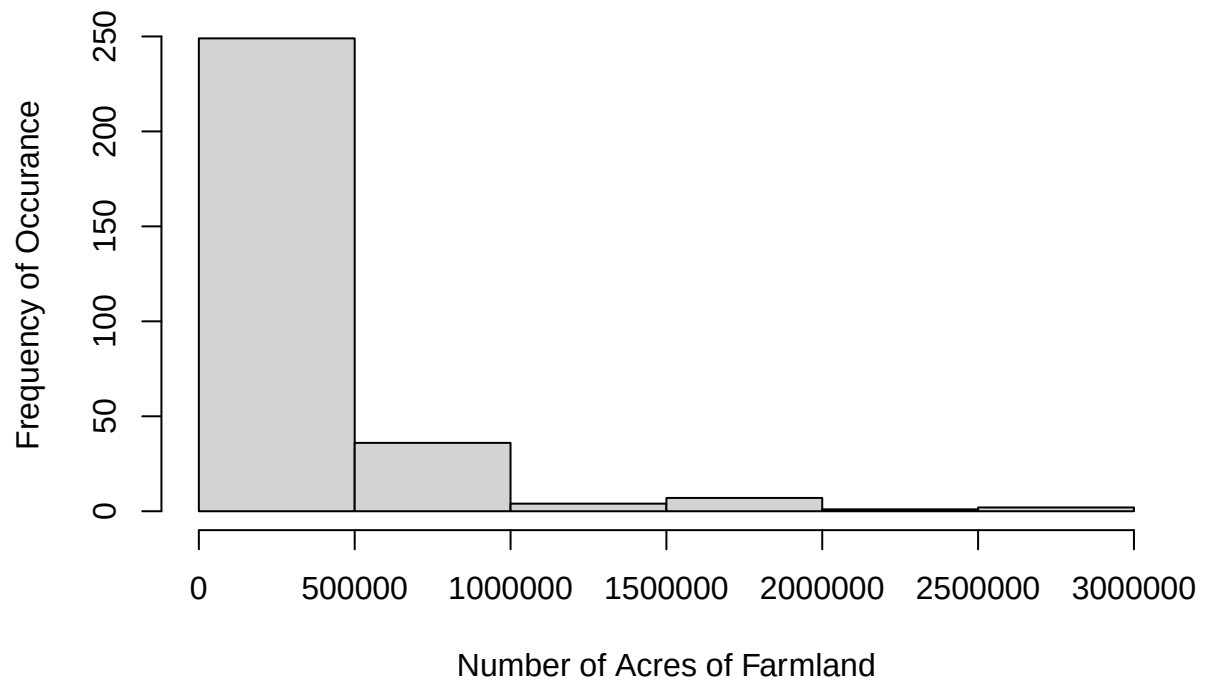
```
## Warning: Removed 1 rows containing non-finite values ('stat_boxplot()').
```



5. Create a histogram of the values for yi from the SRS...

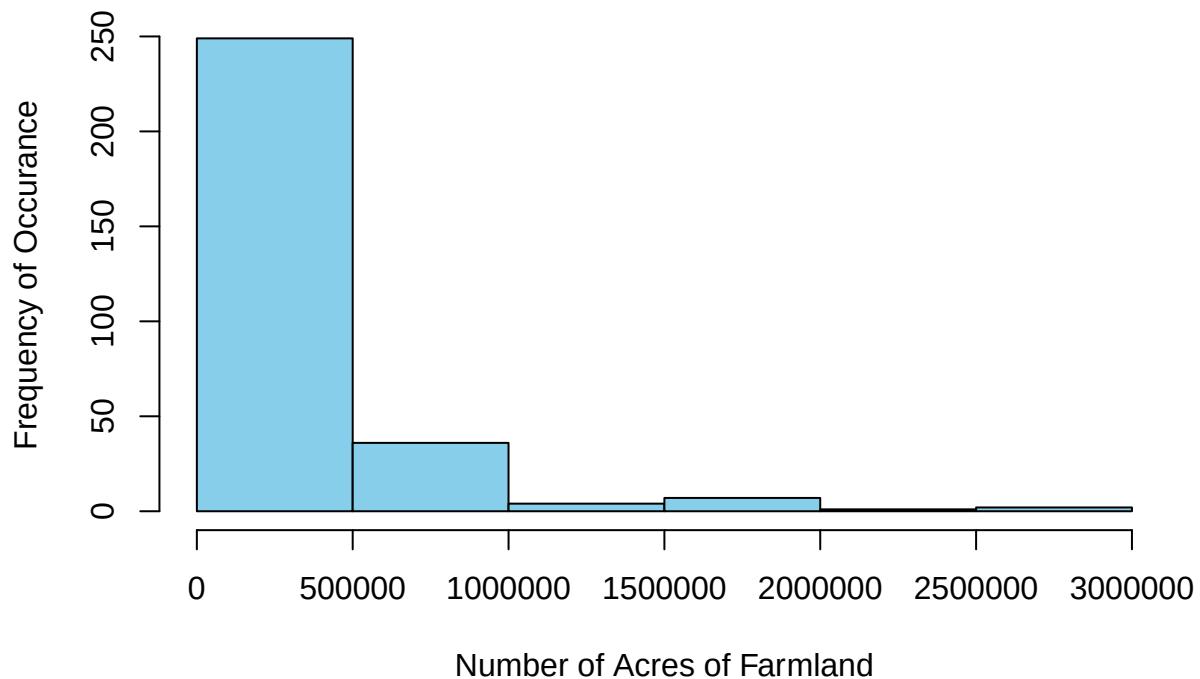
```
hist(srs$acres92, main="Histogram of Farmland in 1992 from SRS", xlab="Number of Acres of Farmland", ylab="Number of Acres of Farmland", y1
```

Histogram of Farmland in 1992 from SRS



```
hist(srs$acres92, main="Histogram of Farmland in 1992 from SRS", xlab="Number of Acres of Farmland", ylab="Frequency of Occurrence")
```

Histogram of Farmland in 1992 from SRS



6. Find the mean and standard deviation of y_i from SRS

```
srs1<-na.omit(srs)
mean(srs1$acres92)
```

```
## [1] 330105.8
```

```
sd(srs1$acres92)
```

```
## [1] 393529.4
```

7. The following code creates a function...

```
conf.mean.fpc<-function(y, N, alpha=.05){ + ybar<-mean(y,na.rm = TRUE) + s<-sd(y,na.rm = TRUE) +
n<-length(y) + tcrit<-qt(1-alpha/2,n-1) + fpc<-1-(n/N) + lower<-ybar-tcrit*sqrt(fpc)/(s/sqrt(n)) + upper<-
ybar+tcrit*sqrt(fpc)/(s/sqrt(n)) + return(list("Lower limit"=lower, "Upper limit"=upper)) }
conf.mean.fpc(srs1$acres92, N=3078)
```

8. We can use the `t.test()`...

```
t.test(srs1$acres92, conf.level=.95)$conf.int
```

```
## [1] 285318.2 374893.3
## attr(,"conf.level")
## [1] 0.95
```

9. Find a 95% CI for the total number of acres devoted to farmland

```
lower<-length(srs1$acres92)*309730
lower
```

```
## [1] 92609270
```

```
upper<-length(srs1$acres92)*439106
upper
```

```
## [1] 131292694
```

10. Suppose we are interested in the proportion of counties in 1992 that have more that...

```
counts<-ifelse(srs1$acres92 > 350000, 1, 0)
tot.count<-sum(counts, na.rm=T)
tot.count
```

```
## [1] 92
```

11.

12. `conf.prop.fpc<-function(x,n,N,alpha=.05){`

- `phat<-x/n`
- `zstar<-qnorm(1-alpha/2)`
- `fpc<-1-(n/N)`
- `lower<-phat-zstar*sqrt(fpc)*sqrt((phat*(1-phat))/n)`
- `upper<-phat+zstar*sqrt(fpc)*sqrt((phat*(1-phat))/n)`
- `return(list("Lower limit"=lower, "Upper limit"=upper))`
- `} conf.prop.fpc(tot.count,300,3078)`

12. We can use the `prop.test()` function to find a confidence interval for the proportion without the `fpc`:

```
prop.test(tot.count, 300, conf.level = .95,correct=FALSE)$conf.int
```

```
## [1] 0.2572058 0.3610162
## attr(,"conf.level")
## [1] 0.95
```

13. Select a stratified sample of size 300. For this we use..

```

agpop1<-na.omit(agpop)
h1.1<-subset(agpop1$farms92,agpop1$region=="NE")
h1.1samp<-sample(1:NROW(h1.1),21)
h2.2<-subset(agpop1$farms92,agpop1$region=="NC")
h2.2samp<-sample(1:NROW(h2.2),103)
h3.3<-subset(agpop1$farms92, agpop1$region=="S")
h3.3samp<-sample(1:NROW(h3.3),135)
h4.1<-subset(agpop1$farms92,agpop1$region=="W")
h4.1samp<-sample(1:NROW(h4.1),41)
df2<-cbind(h1.1samp,h2.2samp,h3.3samp,h4.1samp)

```

```

## Warning in cbind(h1.1samp, h2.2samp, h3.3samp, h4.1samp): number of rows of
## result is not a multiple of vector length (arg 1)

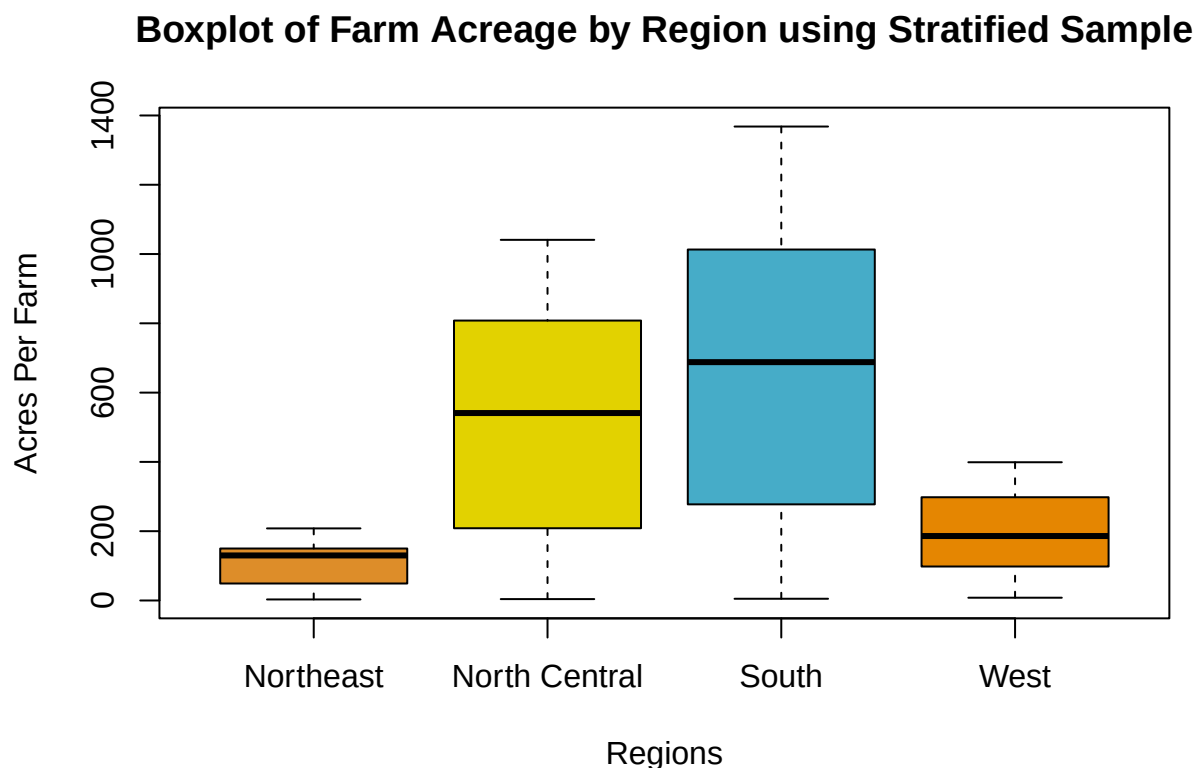
```

14. For the stratified sample plot...

```

boxplot(df2,names=c("Northeast","North Central","South", "West"), xlab="Regions",ylab="Acres Per Farm",

```



15. The following code creates a function that calculates....

```

strat<-function(data,Stratum, Nh,nh, alpha=.05){ m<-tapply(data[,1],data[,2],mean,na.rm=T) v<-
tapply(data[,1],data[,2],var,na.rm=T) y<-m*Nh z<-Nh^2*(1-nh/Nh)*v/nh y<-cbind(m,v,y,z) row.names(y)<-
Stratum colnames(y)<-c("y(bar).h", "s^2.h", "t.hat.h", "Est. Var. of. Tot") t.hat.str<-sum(y[,3]) var.hat.t.hat.str<-
sum(y[,4]) SE<-sqrt(sum(y[,4])) y.bar.str<-t.hat.str/sum(Nh) var.hat.y.bar.str<-(1/(sum(Nh)^2))var.hat.t.hat.str

```

```
SE.y.bar<-sqrt(var.hat.y.bar.str)  zstar<-qnorm(1-alpha/2)  tstar<-qt(1-alpha/2, sum(nh)-length(Nh))
lower.z.t<-t.hat.str-zstarSE upper.z.t<-t.hat.str+zstarSE lower.t.t<-t.hat.str-tstarSE upper.t.t<-t.hat.str+tstarSE
lower.z.y.bar<-y.bar.str-zstarSE.y.bar  upper.z.y.bar<-y.bar.str+zstarSE.y.bar  lower.t.y.bar<-y.bar.str-
tstarSE.y.bar  upper.t.y.bar<-y.bar.str+tstarSE.y.bar  return(list("Summary"=y, "t.hat for stratified"=
t.hat.str, "var.hat(t.hat) for stratified"=var.hat.t.hat.str, "SE of t.hat for stratified"= SE, "y.bar for
stratified"= y.bar.str, "var.hat(y.bar) for stratified"=var.hat.y.bar.str,"SE of y.bar for stratified"= SE.y.bar,
"Lower limit for 100(1-alpha)% CI for t using the Normal Distribution"=lower.z.t, "Upper limit for 100(1-
alpha)% CI for t using the Normal Distribution"=upper.z.t, "Lower limit for 100(1-alpha)% CI for t using
the t-Distribution"=lower.t.t, "Upper limit for 100(1-alpha)% CI for t using the t-Distribution"=upper.t.t,
"Lower limit for 100(1-alpha)% CI for y.bar.U using the Normal Distribution"=lower.z.y.bar, "Upper
limit for 100(1-alpha)% CI for y.bar.U using the Normal Distribution"=upper.z.y.bar, "Lower limit for
100(1-alpha)% CI for y.bar.U using the t-Distribution"=lower.t.y.bar, "Upper limit for 100(1-alpha)% CI
for y.bar.U using the t-Distribution"=upper.t.y.bar))

strat(df3[c(1,2,3,4),], Stratum=c("NE","NC","S","W"), Nh=c(220,1054,1382,422), + nh=c(21,103,135,41))
```

16. Compare the results for the stratified sample with those from the SRS: The SRS displayed more outliers on the boxplot than the stratified sample. This gave a higher amount of variance for the SRS than the stratified sample. The stratified sample showed less outliers and better grouping, and so should have less variance, however this shifted the mean for two of the regions. In terms of CI or mean estimation the stratified better represented the regions but did not represent the whole population as well as the SRS.

17. Proportional allocation was used in the stratified sample above:

18. 18. Select a stratified sample of size 300 from the data in the file.... Need to see how my code changes will change the code for this. Do I need to redo each one of the h1.1samp variables and then run my code the way I did to get my previous results? Or since it seems to be wrong due to the output given for question 15 and the given code there, do I just need to try again.

The problem is when attempting to put in the code as given I received errors when trying to use the nrow function and was only able to get the code to go through without error by using NROW, but that seems to have messed everything else up. Specifically,

This is my code as carried over from what was on the lab sheet,

```
h1<-subset(farm, region=="NE") Error in subset(farm, region == "NE") : object 'farm' not found
```

Got this initially so I changed to this:

```
h1<-subset(agpopfarms92, agpopregion=="NE") h2<-subset(agpopfarms92, agpopregion=="NC")
h3<-subset(agpopfarms92, agpopregion=="S") h4<-subset(agpopfarms92, agpopregion=="W")
strat.samp<-rbind(h1[sample(1:nrow(h1),21),], h2[sample(1:nrow(h2),103),], + h3[sample(1:nrow(h3),135),],h4[sample(
```

This was the error I got Error in 1:nrow(h1) : argument of length 0

I tried adding the drop="FALSE" argument to each of the h* but it resulted in the same error

If I tried to take a sample from the subset I got this error h1.samp<-sample(1:nrow,(agpopregion == "NE"),21)Error in 1 : nrow : NA/NaN argument IsthisbecauseIhavecalledtheagpopregion=="NE" argument again? Should I be calling h1 here instead since that is the subset? Either way, the code that I ended up using calls like correctly, where h1.samp<-sample(1:nrow(h1.1),21) where it only calls the subset but it was still resulting in the same errors. I had to use the NROW argument to get this to work.

On top of that, I was also having issues with the rbind and had to switch to cbind to be able to have the subsets have the correct format.

Wait....so...when I used cbind I used it to make a data frame. And according to the data frame I have 135 observations of 4 variables....uhhhh....hmmmm. Ok..so that is part of the problem. I will continue to work and try to stop by during office hours to maybe hopefully get some light shed on where I went so horribly wrong, specifically what I was doing wrong to get errors with the supplied code...unless I was supposed to do something on my end and failed to do.